

Trabajo Final – Cliente de Correo Electrónico

Cátedra: Estructuras de Datos en Python UNaB –

Comisión 2

Integrantes: María Lara Zobaji y Mauricio Ramírez

SCRIPT PRESENTACIÓN:

LARA:

Nuestro proyecto es una simulación de un sistema de correo electrónico.

Donde tenemos desde que el usuario que escribe el mail hasta la red de servidores que hace que podamos enviar un mensaje de un usuario a otro.

Para este proyecto usamos Python, la programación orientada a objetos y los algoritmos de grafos como base principal.

Tenemos que tomar en cuenta que el sistema está dividido en 4 componentes claros que son

- Usuarios: Quien envía y recibe mensajes.
- Mensajes: Contiene toda la información como remitente, cuerpo y sus etiquetas para saber la importancia que tenga ese correo, o sea si es urgente o no.
- Carpetas: Con las que organizamos los mensajes
- Y las redes: Servidores de Correo y Red de Servidores que son nuestros grafos.

Cada uno de estos cumple un rol específico dentro del sistema.

Para nuestro trabajo usamos POO porque podemos repartir las responsabilidades de cada clase y que tiene que hacer de manera más eficiente.

Implementando interfaces como envió y receptor que garantizan que el usuario pueda enviar y recibir mensajes.

Bueno ahora vamos a explicar más sobre grafos y la búsqueda recursiva porque: ¿Como podemos buscar un mensaje dentro de una carpeta y otra carpeta sin perdernos?

Con el método buscar_mensaje que lo que hace es revisar su propia carpeta para después llamarse a sí mismo en cada subcarpeta.

MAURICIO:

Para que este cliente de correo sea eficiente y escalable, implementamos tres estructuras de datos fundamentales:

Un Árbol General para las carpetas, una Cola de Prioridad para los mensajes urgentes y un Grafo para la red de servidores.

Árbol gestión de carpetas:

- Para organizar los correos, usamos un modelo de árbol general recursivo, donde cada carpeta es un nodo que puede contener dentro otras subcarpetas.
- Código modelos/objetos.py: en la clase Carpeta usamos un diccionario subcarpetas para manejar los hijos. Permitiendo una búsqueda rápida.

(Demo en vivo): Usamos la opción 5 de buscar mensaje.

Al buscar, el sistema recorre recursivamente mi bandeja de entrada y cualquier subcarpeta que haya creado.

Aunque solo la llame una vez, “el código se llama a si mismo” para profundizar en la jerarquía.

Complejidad:

Esta búsqueda recursiva tiene complejidad de $O(N)$ (es lineal) ya que en el peor de los casos tenemos que visitar todas las carpetas para asegurarnos de encontrar el mensaje

Cola de prioridad:

En un servidor de correo real, no se procesa todo por orden de llegada, sino que los mensajes urgentes deben tener prioridad y mostrarse primero, para eso usamos el Heap en el código (dominio/clases.py).

En la clase ServidorCorreo utilizamos la librería Heapq. Así cuando llega un mensaje si es urgente, el método `__lt__` de la clase mensaje le da prioridad, colocándolo adelante de la cola automáticamente.

(Demo en vivo): Luego de enviar correo ya sea con la opción 1 o 2 y rellenar los datos, al final cuando pregunta

¿Es urgente? Respondemos Si para que así el sistema lo trata con mayor prioridad que un mensaje no urgente.

Complejidad: La gran ventaja acá es la eficiencia. insertar y sacar mensajes de la cola prioridad nos cuesta $O((\log N))$ es logarítmica) manteniendo el servidor rápido sin ralentizarse por muchos correos.

Grafo de Servidores:

-La clase RedServidores modela la infraestructura. Cuando conectamos dos servidores, creamos un nodo y una arista en el grafo. Esto permite que el siguiente integrante use los algoritmos de ruta (BFS/DFS) para simular el envío de correos a otros dominios.

En el código: (Grafos/grafos.py): nuestra clase RedServidores usa una lista adyacencia. los servidores son nodos y las conexiones son aristas.

LARA:

En cuanto al enrutamiento elegimos dos opciones que pueden ser seleccionadas a la hora de enviar un mensaje: (Enrutar es el camino que toma el mensaje)

BFS: Explora el grafo por capas primero por los más cercanos y luego los más lejos, pero encontrando todo con menor cantidad de pasos

DFS: Explora hasta el final de un grafo para después volver y recorrer otro camino, y así sucesivamente hasta que pueda encontrar lo que necesita.

Así se pueden comparar entre sí y sus diferentes estrategias de enrutamiento.

Ahora, tenemos las funcionalidades avanzadas en nuestro sistema que permiten al usuario mandar mensajes, definir que red de ruta de viaje utilizaran y cuál será su prioridad.

MAURICIO:

(Demo en vivo): Usamos la opción 1 y creamos 2 servidores, Servidor A y Servidor B, luego usamos la opción 2 para conectarlos. al enviar un correo desde un usuario de A hasta un usuario de B, el sistema usa un algoritmo de búsqueda (BSF o DFS) para encontrar el camino. Al enviar un mensaje nos saldrá el aviso de que ruta siguió el mensaje a través de los nodos del grafo.

Complejidad: para encontrar la ruta usamos BFS (búsqueda a lo ancho), que tiene una complejidad $O(V+E)$ (vértices + Aristas) asegurando que encontramos el camino más corto y rápido.